

- (a) **Top-Down, First-Match Traversal** The system starts the mapping at the root of the application tree. It uses our defined interpretation function f to select the child node with the highest cosine similarity to the user input. The process repeats until a leaf node is reached, moving to children nodes only after the parent node is classified.
 - (b) **Early Stopping for Efficiency & Error Mitigation** A threshold is in place for cosine similarity. If the similarity between the user input and the best-matching node at any level is below a predefined threshold, the search is halted, and the system can proceed with action execution based on the current best match, or prompt the user to try again. This optimizes the search process, ensures responsiveness and mitigates errors that can result from forced mapping.
2. **Conditional Logic** Based on the analysis, the algorithm employs conditional logic to deploy appropriate agents in the extraction mechanism. Parent nodes of classified applications are checked, to guarantee the most suitable model is utilized. This selection process is pivotal to the algorithm’s adaptability and accuracy in parameter extraction.
3. **Response Generation** Depending on the categorization:
 - If the input falls into a specific category (e.g. calculations), the algorithm engages a dedicated module designed for that category, employing specialized deep-learning models, such as our custom fine-tuned Google’s T5
 - For other types of input, the algorithm utilizes a different set of models tailored for general language understanding and response formulation
 - This delegation of tasks makes it so that we use the most appropriate model for the task, employing larger more capable models for more demanding tasks such as complex arithmetic extraction, while making use of suitable smaller models for simplistic tasks. This optimization ensures valuable inference time from larger models is not wasted on tasks that don’t require it, saving resources along with crucial response time.
4. **JSON Output Formatting** Finally, the parameters extracted by the model are formatted into a structured JSON object, optimized and suitable for further processing or for display to the end-user once sent back to the client. *See Listing 1.3.*

Listing 3: Extracted parameters parsed into a JSON object

```
{ "CurrentApp": "AccountForm", "Config": { "Name": "Connor Syle", "Address": "34
Coronation Street", "Email": "connor32@outlook.com" } }
```

This pipeline employed by our LLM engine allows for complex comprehension of textual input from a user, by accurately classifying and extracting parameters that the target application requires. Our engine is able to streamline the connection between the user and the software by accounting for the intentions and requirements of both, utilizing its logical reasoning to complete the task as it sees fit. This is a stepping stone in the right direction for the next generation of software.

5 Evaluation

This section intends to evaluate and validate the abilities of several large language models tasked with serving as the crux of our research. Accuracy in parsed inputs is the driving factor behind this project functioning well. The engine must correctly classify which task the user’s prompt corresponds to, extract, parse, and output the expected parameters in the required JSON format.

5.1 Required Tasks

Classification is accomplished by encoding descriptive nodes for each task along with the input node using an all-MiniLM sentence-transformer model and then selecting the node with the greatest cosine similarity. This classification

GPT Baseline VS Custom Engine Prompt Evaluation			
GPT-4 Generated Inputs	Truth Intention	Parser Output	Eval
I'm registered under the name Alex J. Turner, but everyone sends their regards to my place at 768 Rolling Rock Street, and for a quicker response, they hit me up at alex.turns@rocknmail.com.	{ "CurrentApp": "AccountForm", "Config": { "Name": "AlexJ.Turner", "Address": "768RollingRockStreet", "Email": "alex.turns@rocknmail.com" } }	{ "CurrentApp": "AccountForm", "Config": { "Name": "AlexJ.Turner", "Address": "768RollingRockStreet", "Email": "alex.turns@rocknmail.com" } }	✓
Could you tell me if I need an umbrella for my walk today around the canals of Amsterdam, Netherlands?	{ "CurrentApp": "Weather", "Config": { "City": "Amsterdam,Netherlands" } }	{ "CurrentApp": "Weather", "Config": { "City": "Amsterdam,Netherlands" } }	✓
Just finished a book set in Venice, Italy, and it got me wondering about the weather there today. Is it as sunny as in the story?	{ "CurrentApp": "Weather", "Config": { "City": "Venice,Italy" } }	{ "CurrentApp": "Weather", "Config": { "City": "Venice,Italy" } }	✓
My friend in Sydney mentioned they were having quite the heat-wave. I'd love to compare that to the current temperatures where the Great Pyramids stand. Could you give me a quick overview of the weather there?	{ "CurrentApp": "Weather", "Config": { "City": "Egypt" } }	{ "CurrentApp": "Weather", "Config": { "City": "theGreatPyramidssand" } }	×
I've got 24 cupcakes, and I need to divide them evenly among my 6 friends. How many does each person get?	{ "CurrentApp": "Calculator", "Config": { "promptSequence": "24/6" } }	{ "CurrentApp": "Calculator", "Config": { "promptSequence": "24/6" } }	✓

Table 2: Comparison of GPT-4 generated inputs with our custom multimodal LLM engine consisting of custom fine-tuned Google T5 & BERT-Large-Uncased-Whole-Word-Masking-Squad2 outputs. The model performs with extreme efficacy on descriptive and lengthy prompts.

task is extremely reliable and the chance of failure is observed to be negligible. The critical task to evaluate is the parameter extraction once the input's corresponding application has been established. Extensive experimentation and trials were conducted with plentiful transformer models from the Hugging Face hub. Models specialized in Question Answering as well as Text2Text Generation were found to perform extremely well.

5.2 Model Selection

For this evaluation, we pit the most effective models found during initial testing, with available resources for training, hosting, deployment and inference in mind—the T5 Text2Text Generator models along with BERT and ELECTRA. The BERT and ELECTRA models were already fine-tuned on the SQuAD 2.0 dataset, which is a popular dataset used in fine-tuning models for logical reasoning and question-answering. On the other hand, the T5 model base and multi-task, trained by Google, were also chosen due to their flexibility, scalability, and efficacy in abstraction and multi-task situations.

5.3 Observation

We can see from the benchmarks in *Figure 6* that the T5 model trained on specialized custom datasets relevant to specific application tasks does exceptionally well across all tasks. It performs the best in three out of four tasks, with the last one being a close tie. The BERT model also performs very well but struggles with the logical capabilities required for the advanced calculator prompts. Google's multitask-trained model falls extremely short in more logically involved

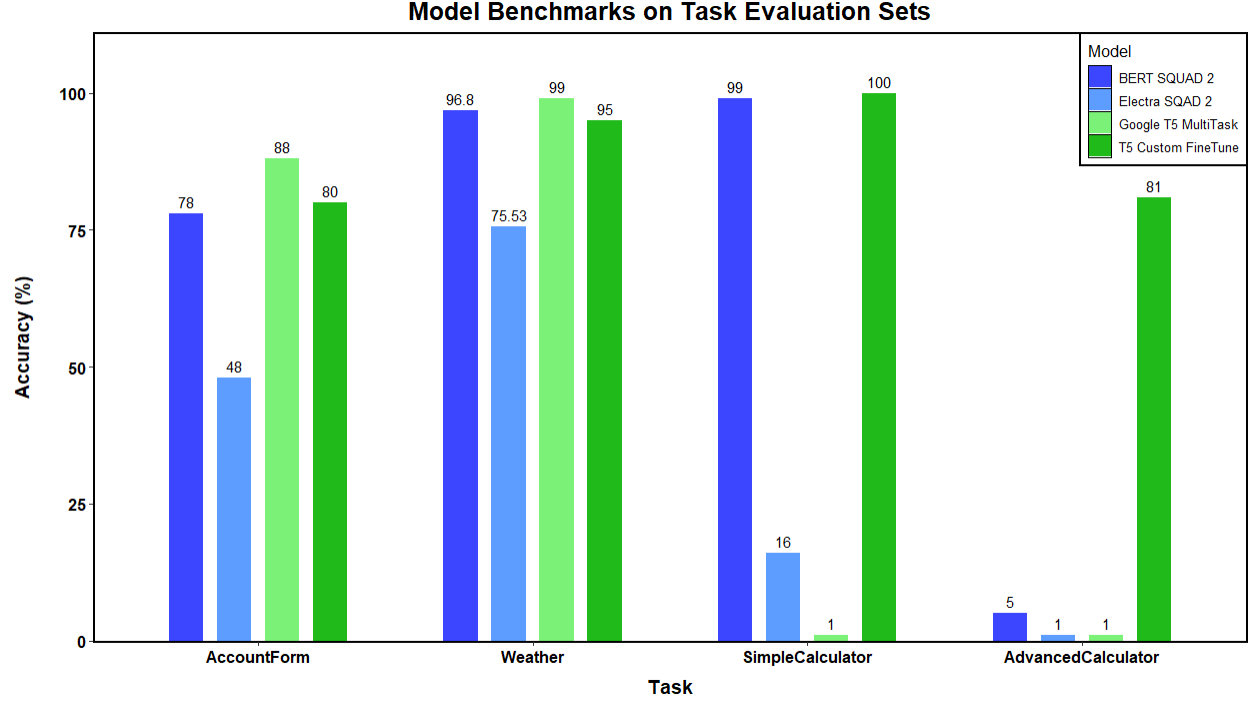


Figure 6: Accuracy comparison between models engineered for our application’s tasks. Each model was trained on logical and information extraction tasks, and further carefully prompt engineered to achieve the best performance possible for each task.

tasks, while the ELECTRA model does poorly overall. Further training of the T5 model with more diverse and better quality data would allow it to be the most versatile and powerful model for our framework.

6 Future Directions

To improve this framework and scale its potential, continuous iterations on both front-end and back-end engine components need to be considered.

6.0.1 Automation of Annotated UI Paradigm

An automated structure for generating classical UI components through textual descriptions would allow for a unified annotated system. If a developer’s textual prompts can generate UI components, they can be parsed to serve as annotations for the components as well. This would immediately make the UI compatible with the LMM engine’s requirements and would transform the way full-stack applications are developed.

6.0.2 LLM Engine Embellishments

Exploring configurations regarding transformer models that classify, extract, and parse information in the LLM engine will improve accuracy and inference speed. The choice between delegating tasks to different models versus using a singular multi-task model for all tasks is open to be explored.

6.0.3 AutoGen

Expanding on the back end embellishments, making use of Microsoft’s AutoGen [11], Mistral’s Mixtral 8x7B mixture of experts model [12], or Ollama’s model hot-swapping to allow for multi-model agent setups may be the ultimate solution for delegating a task to the appropriate model. AutoGen’s optimized mixture of experts system would pose many benefits over a manual swapping system, especially with the capability for agents to communicate with one another. If an AutoGen setup can delegate a task to the best-suited LLM amongst each other, accuracy can be maximized across all tasks and inference times can be further optimized.